

Spring 2026 PACWAR Report

Austin Yilmaz - writing the report & engineering the algorithm
Saurav Kumar Gupta - coding and commenting the algorithm

1 Introduction

In the COMP440 Artificial Intelligence course, we were given a project called the PACWAR project, which sounded frightening at first. The professor was talking about genetically altering 'soldiers' to fight better, and writing programs to find the best gene to make them the best fighters - 'killers' per se. At first glance, it seemed like a really hard project to do just from this description, because we did not know what were the methods to do what was required to engineer our soldiers.

Then, our professor started mentioning search spaces and how one would find a way through them, to get to a goal, find an objective. This is where things started taking shape. Because she also told us that the genes were composed of 50 digits, each digit being in the set $\{0, 1, 2, 3\}$, and each n -digit part of the gene affecting how our soldiers - the so-called PACmites - behave in different ways. When we first looked into the problem, we immediately linked the idea of finding the best gene to a search problem in a vast space of size 4^{50} . However, we were still not sure on how we would navigate through this space...

2 Our High Level Progress in Building Genes

Gene I: Our professor first gave us a task, on just finding a gene that beats the 'all ones' and 'all threes', which pushed us on devising an approach that would not get to the 'best gene', but just one good enough that could beat the 'all ones' and 'all threes' setups. In that same week, she discussed about **genetic algorithms (GAs)**, which operated on 'genes' by combining parts of good genes - with respect to a fitness function - and then crossing them over, just like how breeding works in nature. We thought, we might have finally found our approach! Even the definition fit perfectly to our problem of finding a gene that beats the given setups. We then implemented a simple genetic algorithm (GA) in C, which we will delve into the dynamics more in the later sections, that was fairly quick to find a gene that beat both 'all ones' and 'all threes'. We then sighed, with the pride of overcoming the task of beating the seemingly strongest genes in the whole search space - we were **terribly wrong**.

Gene II: Some time passed, and our professor made us realize more and more through the hints she gave in the lectures that the real task was not to beat the 'all ones' and 'all threes', but to compete against each other in the final tournament. Also, after she published the results of the Gene I tournament, we realized that neither 'all ones' and 'all threes' were good genes, nor the gene we found was a strong contender... Also, the Gene II assignment was different than the previous one; we had to report on our approach of finding good genes for PACWAR, forcing us to 'up our game'. It was then we realized that our evaluation metric for the first gene only depended on "being able to beat 'all ones' and 'all threes'", and that we did not exploit how the parts of the gene influenced the movements and behavior of our PACmites. We built an improved genetic algorithm, which had an evaluation function more complex and similar to what was given in the Canvas page - but still simpler than that; and incorporated a mini GA on the specific gene parts in addition to the general GA we were performing, which we will explain in more detail in the following parts of the report, just like any other approach we will cover in this section.

Gene III: We were not that pleased when the results of the Gene II tournament came in. We saw that our gene was one of the worst contenders in the whole tournament, even though in our GA, it came out right on top! We realized that our GA was nowhere near a good one, nor was our simply 'inspired from Canvas' evaluation metric. We realized that we had to adjust the whole pipeline, not just our evaluation metric or doing local search on gene fields. Then we carefully re-read the Canvas description for the project, and saw that actually most of the concepts involved in the PACWAR game closely relate to the concepts we learned in class, for which they could be used to improve our GA - evolve it to one that would find us a much better gene, for which it did. The gene it found decimated all of our previous genes in a fairly low amount of rounds! Having gained back our confidence, we submitted this gene as the Gene III of our project.

Final Gene: Yet another time, we were very disappointed with the results of a PACWAR tournament, as our Gene III was close to last place again! But it had performed so good in our GA and our tests, how was this possible? It was then we went to ask the professor on why this was happening twice in a row... She answered: "You have

found a local optimum! You need to change how you are evaluating your genes, and how much stochasticity is involved in your GA.'". It was then something in us switched. This whole time, we were forgetting the problem we are dealing with was no ordinary search problem with a single peak, but one with billions and trillions of them. And the approach we were using, GAs, just yielded us local optima, any one of those peaks! The evaluation metric we used for Gene III was too simple to be good! We then moved on to improving our GA one last time, to hopefully yield us the **killer gene**. We increased the randomness of our code, while still keeping the good ones by elitism. We changed our evaluation metric to use a hybridized approach that would both incorporate what was available in Canvas, but also weed out the bad genes sufficiently (do not worry, we will discuss all of these in the following sections). And voila! It yielded us a gene that was drastically different than any of the genes we found earlier digit-wise, and it beat ALL of our previous genes in less than 200 steps! We had finally pulled ourselves out of the local optimum we were stuck in for the past 2 genes. Looking at the screen, the long way we have come, and what we have learned through the progress (which also helped us in different fields - like the research Austin is doing); we smiled, as we were more confident about finding a gene fit for the final tournament than ever.

3 Complexity of the Problem: Man VS. Machine

As mentioned in the introduction, the PACWAR problem given to us had an enormous gene space which was infeasible to explore with a modern PC, and the gene space had a topology full of spikes and dips, which made the problem even harder to solve using only computational power and local search algorithms. This is where human intuition, and domain knowledge came into the picture.

We knew that we needed some kind of grasp of the search space, and to get that grasp, we first needed to understand how the game mechanics work. We analyzed the source code to discover that the game is deterministic in nature, but the stochasticity is introduced by the discontinuous/discrete giant search space that is 4^{50} genes, so we did need some compute to generate random genes to some extent, and also to actually determine if the gene we found is good or not. But to make this compute easier, we needed to steer the algorithm (the genetic algorithm) we would be using to perform the computations, and the steering wheel would be some evaluation function specifically engineered for the game by a human (us).

Too generic of an evaluation function would fail as there would be many promising genes, failing to branch and bound the search space, as we have seen in our Gene 3 (zero-sum eval function); and too specific of an evaluation function would prune away really good genes that just were bad in some aspects, as we have seen in our Gene 1 (ones-threes eval function).

In summary, this computational complexity and the need for human engineering tradeoff demonstrates the cruciality of the need of domain knowledge of the PACWAR game. One famous quote by Donald Knuth is 'Computers are good at following instructions, but not at reading your mind.'. So, we should be careful on what instructions we give to the computer, and how well it represents what's going on in our minds. **(Another example would be the professor's quote on how it is not the intelligence of the AI agent that may be dangerous to us, but us giving the agent a wrong model of the space/a wrong objective to optimize.)**

4 Process of Finding and Evaluating Each Gene

4.1 Gene I: A fresh start

Given the task of beating 'all ones' and 'all threes', and having learned what GAs are, we implemented a simple GA, that worked through the following pipeline:

1. Generate a random population, with two of the genes in the population being 'all ones' and 'all threes' with special labels.
2. Duel the genes in a tournament fashion, with 'all ones' and 'all threes' being in at least half of the duels, and reward them in the following manner:

- (a) Win against normal gene - Small reward
 - (b) Win against 'all ones' OR 'all threes' - Huge reward
 - (c) Lose against normal gene - Small penalty
 - (d) Lose against 'all ones' OR 'all threes' - Moderate penalty
3. Cross the best genes found (except 'all ones' and 'all threes') over and mutate to generate the new population.
 4. Keep doing the same until a gene beats both 'all ones' and 'all threes'.

In designing this algorithm, our expectation was that it would find a gene that beat 'all ones' and 'all threes', which we presumed were one of the best genes in the whole search space. That is why we introduced such a large reward for winning against 'all ones' or 'all threes'. The rest was implemented using the high-level explanations given in the lecture notes, so it was just a typical GA with a 'funny' evaluation metric specifically designed for our task.

The algorithm converged fairly fast, and it did find a gene that beat both 'all ones' and 'all threes'. This result was to be expected, as we had heavily rewarded beating any of those two compared to the normal duels we did. We were confident that we were able to solve this problem this quick. We were like, 'the rest is going to be a cake walk'. Again, **we were terribly wrong**.

4.2 Gene II: 'Real'izing the 'Real' Objective

For the second gene, the task was to slowly but steadily start on finding a gene that would participate in a competitive environment where it would battle against the genes of our classmates. We then thought, if 'all ones' and 'all threes' were actually good contenders, then why did the scores deviate that much in the first tournament? This indicated that the gene space was filled with billions of peaks, as we had mentioned. We then read the final project specifications more, and have come to the realization that we had not incorporated specific gene field, nor a 'hall of fame' of our killer genes into the GA. We also needed to modify our evaluation function to take into account that 'all ones' and 'all threes' are not the only 'best' genes in the space. We modified our GA to do these in addition to what we were doing for Gene I:

1. We added a local hill climb-like search based on the specific gene fields that influenced the behavior of our PACmites. **This would help us exploit the fact that the reason one gene might be failing might be a result of a specific gene field being bad, not the whole gene! It would also help us combine two good genes by combining the 'good fields' of both - that was our intuition.**
2. We changed the evaluation metric by getting rid of the 'all ones' and 'all threes' reward, and getting inspired from the Canvas project outline, implemented a metric that would treat all genes equally, and had more continuous but smaller rewards - still not that similar to what was in the Canvas page. **We did this because we believed simplicity would lead us to an optimum faster, separating the good from the bad better, for which we were wrong.**

In making these changes, our expectation was that this GA would yield the 'killer gene' for PACWAR. We believed that our simple evaluation metric coupled with gene-field search would only keep improving our genes, for which we were mostly wrong. Even though the local search on specific gene fields helped us get better genes, the improvement got stuck after a while. We mistakenly took this for 'having found the best gene' and submitted it. What we did not realize was how many local optima exist in this search space and that a simpler evaluation function in a GA does not mean a better optimum in general.

The algorithm converged fast, just like Gene I. We had not expected this result, as we thought the algorithm would just keep improving the genes more and more as it got closer to the 'killer gene'. But the real reason was the algorithm just kept stagnating in a local optimum! We would not come to realize this until speaking to the professor after the tournament results of Gene III.

4.3 Gene III: Flabbergasted (not) from Success

After the terrible results of Gene II tournament came in, we decided that maybe not just our evaluation metric or how we were doing the search was the problem, the whole pipeline needed to be improved for the GA to converge to the 'killer gene'. We were slowly realizing that local optima existed in the space, and that we were stuck in one, but we were confident that the number of these local optima was not many, so we were confident in the modifications we did to our GA for Gene III, which were:

1. We changed our evaluation metric to one that was much simpler than our Canvas-inspired one, just +1 for win, 0 for draw, -1 for loss. We did this because we believed that simpler rewarding would increase stochasticity, therefore pull us out of the local optimum. **However, we did not realize it would do the opposite by making a lot of genes that are close to the local optimum we found have really close fitness values.**
2. We also evaluated the best scores and the means in each generation, so that the results we found were more stable and so that we were more confident in the genes we generate. **This maybe would have helped in another evaluation metric setup, but actually did the opposite in our case since it got the fitness scores even closer and separated the local optima even more from the bunch left to explore.**

We were expecting to find a much better 'killer gene', and maybe progress through the vast number of local optima to 'the' 'killer gene', and our algorithm did give us a gene that was able to beat all of our previous ones in a fairly moderate number of iterations. However, the gene string our algorithm found looked pretty much similar to some of our previous best ones; we had not yet realized that this was an indicator that we were still stuck in the local optimum.

The algorithm also converged fast, just like the two previous cases. It was because the algorithm was stagnating in a local optimum, just one different than the one we input to it and one that was better than it. We were still terribly underestimating the number of local optima in this search space! Fingers crossed, we were waiting on the results of the Gene III tournament, to see if we had found the perfect recipe for the potion of love (or the best PACWAR gene)... - we did not.

4.4 Final Gene: Where Everything (Hopefully) Came Together

The results of yet another disappointing PACWAR tournament came. And our place still did not improve, but did not worsen either. What more of an indication did we need to realize that we were stuck in a local optimum? We went to the professor to get some answers on why we were failing so bad, for which she replied the following magic sentences:

"You are getting stuck in a local optimum, because your evaluation metric is not good enough. I personally think that all the previous years' champions are also local optima. You need to achieve stochasticity and improvement by having a good evaluation metric."

There it was! We were doing the opposite this whole time, thinking a simpler and simpler 'zero-sum' evaluation metric would get us to a good gene! We then started brainstorming, and implemented the ultimate PACWAR GA (or at least, the best one we could do), to get our **hopefully-killer gene** that destroyed ALL of our previous genes in less than 100 steps, and that looked much different than what our previous genes looked like. We did the following changes, which were crucial for a different reason each:

1. We aligned our evaluation metric with the one given in the Canvas page, getting rid of our overly simplistic previous evaluation metric. **We realized that getting out of the local optimum was not fast and random, but slow and steady; and building a good evaluation metric - especially one that was centered around what our genes would be tested against - was the key step in ensuring our search progressed.**
2. We replaced single-number average fitness to a more complex objective to reflect on if we are stuck in a local optimum or not. Aside from taking the mean of the fitness approach we were doing in the Gene III setup, (which we thought would help us here even though it made our job worse in Gene III - since the fitness variance was much higher in this current evaluation metric), we implemented other metrics in our evaluation,

including worst-case slice, hard-loss penalties, variance penalties. **This was to punish 'unstable' genes that behaved too good against some strong genes but lost fairly quickly to much simpler weaker genes.**

3. Just like the Canvas evaluation metric, we implemented behavior-sensitive rewards that were tied to the game mechanics. Faster kills were rewarded just like in Canvas, but we also adjusted the rewards on how many PACmites were left after the battle (even though usually it fills the whole arena). We included a control-margin signal that was based on survivor differences. **This was to push the evaluation towards genes that performed faster and more decisive killings, i.e. more consistent and more convincing genes.**
4. We added a local optimum-stagnation detection mechanism to increase randomness to help us get out of the local optimum. If our progress plateaued for several generations, we proportionally upped the mutation scale and added **immigrant counts**, which meant randomly injecting genes in our population. **We believe that this was the most important step we did on progressing through the local optimum escape, as our drastically different 'killer gene' appeared right after our algorithm was stuck for around 30 generations.**
5. We awarded genes that were far from the previous best depending on their Hamming distance by a small amount, another approach to increase the randomness of our genes to help us escape local optima.
6. We also improved the hall of fame selection present in our previous GA for Gene III. We inserted the current champion into the opponent pool of the next generation in a different way than Gene III. We battled it out with the best of the previous generation and only inserted it if it won the battle. **Even though this might seem to decrease stochasticity at a first glance, in fact, empirically, it does not. We believe that it has helped us by NOT populating our opponent pool by local optima at the times we were getting stuck, reducing overfitting.**
7. We kept exploration and exploitation balanced by decreasing elitism compared to the previous GAs of ours, and increased the gene field-based crossover/mutation's presence in the overall crossover/mutation process. **This was another step we took on reducing stagnation in a local optimum, and we also believe it worked by making the opponent pool more random and introducing us to a different variety of genes. The reason we did not remove elitism is that we still believed the way of getting out of the local optimum was actually comparing how the new genes fared against the local optimum we had, which turned out to be an excellent strategy - looking at the 'killer gene' we got from the algorithm.**

We expected this approach to finally yield us a (hopefully drastically) different gene that performed much better compared to all of our previous ones, and it actually did. The fruits of what we have implemented above were tasty and healthy. We got a gene that beat all of our previous genes in fewer than 100 rounds, and the interesting part is, the algorithm only took around 600 generations to find that gene, which was around 10 minutes. After adding our newly-found gene manually to our hall of fame, the algorithm failed to find a better one for more than 5000 generations - even though some fairly different genes got close results; so we decided to call it a day and actually test it against the other best genes both found by the algorithm, and our previous best genes, with some added random genes. It achieved an average score of 19.1 out of 20000 duels, which was astonishing! We confidently submitted that gene as our best PACWAR gene, as it had all the indications we wanted for a strong contender to the final tournament:

1. It was fairly different than all our previous local optima.
2. We had fine-tuned our GA to escape local optima when it stagnated.
3. It beat nearly every example we were able to concoct, and it beat every stable example in that set - stable as in does not get beaten by a 'weaker' opponent after beating a 'stronger' one, strength being proportional to fitness.

What more could we want? It was time to document this leap in the PACWAR report, for which we are doing right now....

5 Tournament Performances of Our Genes and Their Features

Here is the table of how well each of our genes did in the 4 PACWAR tournaments:

Index	Entry	Rank	Score
Competition 1	1302100010112122333311221211212211211220333230300	14	11.47
Competition 2	13020000111122020133212212211213213211302320103311	25	5.88
Competition 3	11120100111122231113233213212222212213211313303310	28	5.06
Final Competition	03100000330000200333321121121121321121221130131131	23	10.03

Table 1: Competition Entries and Performance

5.1 Gene I: 1302100010112122333311221211212211211220333230300

This gene was specifically engineered by us to defeat 'all ones' and 'all threes' in the fastest way possible, using our Gene I code.

It works by repeatedly swarming and sending our PACmites from the middle, which exploits the all ones'/all threes' weakness of spinning in circles to defend themselves and slowly sending out PACmites to destroy the enemy, by a fast shock 'swarm' attack. It beats the 'all ones' in around 455 rounds, and 'all threes' in around 188 rounds. While the 'all ones' and 'all threes' are wasting their time spinning in circles, they are getting swarmed by this gene, which lowers the 'all ones' and 'all threes' defenses, and then sends PACmites to sneak in and perform the kill when the perimeter is breached.

It is an above average local optimum that was derived using an evaluation function based on defeating the 'all ones' and 'all threes', as explained above. The tournament score indicates that it is an average performing gene in the pool, but making a comment using the tournament score as an anchor would not be a perfectly correct thing to do as all the genes were designed to beat 'all ones' and 'all threes' and not actually compete against each other.

5.2 Gene II: 13020000111122020133212212211213213211302320103311

This gene was engineered using an evaluation function inspired by the PACWAR outline, but was simpler (rewarding all wins, losses, and draws the same) and had larger rewards compared to the tournament's evaluation function described in the PACWAR outline.

It works by filling the arena as quick as possible, and then 'pushing' the opponent like a moving wall in an Aztek temple booby trap, annihilating them in the end. However, its defense is not that great, and while it performs really good against defense-oriented genes like 'all ones' and 'all threes', it gets beaten by more aggressive genes that expand in the horizontal and diagonal directions, as rather than defending, it is 'reproduction' oriented, and is prone to attacks itself.

It is an improvement over Gene I, but one can easily see the resemblance in their strings, so it might also be regarded as a local optimum derived using our first gene with just a different evaluation function. The tournament score indicates that it got beaten by most of the other genes (assuming Round-Robin matching), which supports our predicate that it gets beaten by aggressive genes because of its behavior.

5.3 Gene III: 11120100111122231113233213212222212213211313303310

This gene was engineered using an evaluation function much simpler than the first two, which was just a zero-sum evaluation function that regarded wins as +1, losses as -1, and draws as 0; no matter how many PACmites of each gene remained in the arena.

It works by filling the arena up as quick as possible, just like Gene II; while expanding in not a 'pushy' way,

but a 'swarmy' way, in which it resembles Gene I in that aspect. It can be thought as a 'jack-of-all-trades' derivative of the first two genes we have found. It annihilates both aggressive genes by swarming them from behind, and also defensive genes by filling up the arena before attacking. However, it is still vulnerable to horizontally-expanding aggressive genes as it does not send PACmites like Gene I while swarming, meaning the PACmite cluster it forms can be separated from the middle, making it easier to 'capitulate' per se.

It is an improvement that combines the behavioral aspects of both Gene I and Gene II, but it is still in the same spatial locality as them, which implies there is a problem with our evaluation function, and it is getting stuck at local optima (as discussed above). The tournament score indicates that it got beaten by most of the other genes, similar to Gene II, meaning that the vulnerability of Gene II is carried over into this one, just in a behaviorally different way as explained.

5.4 Final Gene: 03100000330000200333321121121121321121221130131131

This gene was engineered using our Final Gene evaluation function, which is described in depth, above. It is a gene that is drastically different both in string composition, and behavior compared to our first 3 genes.

It works by building a thin wall, not expanding behind, but not expanding forward before it is sure it is safe to attack the enemy gene. It will then swarm from both ends, while also sending 'scout' PACmites to ensure the middle isn't being attacked, and 'eat' the enemy soldiers like how PAC-MAN would eat a ghost. However, its attack is not its strongest point, nor its defense. This makes it vulnerable to strong attack-oriented genes. It can be thought as a more 'balanced' character in a video game, like the Knight in Clash Royale. Hence it received a perfect average score of 10.03.

It is a drastic improvement over all of our genes that uses a completely different strategy, but it is still a local optimum as can be observed from the Final Tournament score.

6 Lessons Learned After the Final Results - What to Do If We Had More Time

We have got the chance to listen to our friends' strategies for a brief moment after the final PACWAR tournament. The winning genes have explained their approaches to the problem, and we realized that there is a lot to learn from those explanations.

To improve our gene and get something close to 'harikariberi - the killer gene', we can consider the following two approaches:

1. **The winners' (of 2026 PACWAR) approach to the problem:** We can adjust our evaluation function to take into account mainly not 'how fast a gene beats the other gene', but 'how long it survives against the other gene', because in general, since this space has a topology with many local optima which easily beat other genes, but also easily get beaten by other genes; so it is not a matter of 'when', rather, it is a matter of 'if' we will win the game. This yields much more resilient genes that maximize survival against destruction. This is also much closer to how animals (like humans) evolve.
2. **A larger Hall of Fame of many previous optima:** After some point, evaluating our genes using random genes becomes not that good of an approach because in general, a random string for a gene is a really bad contender at PACWAR. That is why, as we keep exploring new 'killer genes', we need to actively place them into the opponent pool and reduce the 'complete randomness' of our opponent generation as our Hall of Fame size increases. This way, we have actual good contenders to benchmark against, and will not reward hack an 'above average' gene to oblivion using really bad random generations.

This way, we believe that many better genes than all of our tournament entries can actually be generated, and the stagnation in gene performance/killergene generation will be overcome by the Hall of Fame policy.

7 Conclusion and What We Learned

All in all, we think that the PACWAR project taught us a bunch of different things, that helped us both in the span of this course, but also strengthened our intuition and helped us develop new, general approaches to the problems we face in our academic life, such as, but not limited to:

1. **Determining the type of problem you are dealing with is key to solving it properly.** We had first misregarded this problem as a 'convex' global optimum search problem, which had one single 'killer gene'. But then, as we kept exploring and encountering more and more local optima, we realized that this was a problem where we were trying to find the highest peak we could in a landscape of gazillion peaks, and the methods we needed to solve this problem involve some kind of stochasticity as it is extremely easy to get stuck on a peak if not.
2. **Not everything is bad about a bad solution of the past.** In the first three iterations of our GA, we just switched approaches drastically to solve the problem, not paying attention to the actual good aspects of our previous GA codes, such as the not-so-simple evaluation function of Gene II GA, or the local gene-field based search of Gene II GA. After realizing these were actually improvements even though the solution yielded was not that good, we decided to incorporate these and even build upon these (such as the unstable gene punishment mechanism derived from the Gene III GA) in the Final Gene GA.
3. **Some dumb approaches might actually produce really good ones.** While developing our Final Gene GA, we considered getting inspired from something that was totally out of our lecture notes, a mobile game! We got inspired from Clash Royale - a mobile game where in a symmetric arena, blue and red kings battle - and did each of our duels symmetrically. We decided to evaluate each duel twice, by first assigning a gene as Red/first and then as Blue/second in the same duel setup. Because in Clash Royale, depending on whether you are the 'true red/true blue' king, some actions might have different outcomes. We then checked the source code of PACWAR, and realized that being the 'blue' or 'red' gene does not matter at all. But we used this idea to implement a whole new paradigm derived from 'difference in similarity', which was to award genes based on Hamming distance to get out of local optima, and that actually worked really well, as our Final Gene GA kept finding drastically different genes with similar good scores!
4. **Winning the war is more important than winning a battle.** The genes we developed focused on beating the opponents in the fastest and most lethal way possible, but they fared bad against the genes that exploited their weaknesses, especially the ones with good defensive behavior. This has made us realize that in general, **not getting killed** is more important than **killing the enemy**, even if it may lead to a draw. So, we should adjust our evaluation functions in PACWAR (and any game of similar kind) on survival, because we do not know what we are fighting against.

In conclusion, we think that we have benefitted largely from this course project, as it has improved our intuition on problem solving, how to start concocting a solution, and how to improve upon that solution. We thank everyone that has had a piece in making this project, and our professor on giving us this opportunity to test our problem solving skills by introducing us to a unique problem with such a large search space, **one that seems completely stochastic but has a deterministic framework underneath; a framework which cannot be fully explored using the computation we have at hand, one that requires intuition. That intuition, we believe we have acquired along the way.**